業務主権を支える実装交換可能性

AI 支援ソフトウェア開発方法論の探索的実証

神 敬二

研究論文草稿 第 1.1 版　2026 年 11 月 7 日　未査読

要旨

AI によって別の実装を作りやすくなっても、組織が安全にシステムを交換できるとは限らない。本研究は、業務の意図、明示的な交換契約、版を特定した実行証拠を継承資産とし、実装交換を条件付きの工学的判断として扱う「業務主権開発」を提案する。構成的な設計変更申請事例を対象として、別々に実装した JavaScript および Python の API を検討した。提供されたレビューは、従来の受け入れ評価が各実装 21 件合格していたにもかかわらず、Unicode 文字数、日時解釈、真偽値と整数の比較に不一致があり、ローカル保存経路が仕様の件数上限より先に読み出せなくなることを指摘した。これらを契約と実装に還元した記録を分析した結果、両実装は同じ 44 件の評価に合格した。各方向 1 件の小規模移行に対する計 6 アサーションでは、取込みの成功、申請配列、仕様切替監査配列の保持を確認した。さらに、各実装で最大文字数の Unicode を含む 200 件を作成し、別実装への移行を検査した。結果は限定された実現可能性を支持するが、完全な動作等価性、無人再生成、開発費用の削減、AI 提供元からの独立を示すものではない。本研究の貢献は、実装交換の操作的判定条件、反例を仕様へ還元する具体的工程、およびその成立範囲を検査できる探索的成果物である。

キーワード： ソフトウェア進化、仕様駆動開発、実装交換、AI 支援ソフトウェア工学、データ移行、テストオラクル、監査履歴の保持。

1 はじめに

ソフトウェアを保守できることと、組織が業務上の決定を自ら支配できることは同一ではない。業務規則がソース、配信スクリプト、保存形式、会話履歴へ分散すると、実装の交換には、業務の意味と過去の判断の記録を復元する作業が必要になる。ソースが入手できても、別実装が同じ入力を同じ意味で扱い、同じ責任分離を実施し、同じ履歴を保持するとは判断できない。

この問題には既存のソフトウェア工学の系譜がある。System/360 は異なる機種間の互換性を設計した[1]。データ独立性はアプリケーションを保存表現から切り離し[2]、情報隠蔽は変化しやすい設計判断を利用側から隔離した[3]。これらは、業務上の約束に対する決定権を組織に残したまま、実装を選び直すために何を確かめるべきか、という問いへ接続する。ただし、特定の計算機方式が業務の従属を引き起こした、あるいは次の技術が必然的に従属を解消する、という歴史的因果を裏づけるものではない。

コード生成や既存リポジトリの修正が研究課題となったことで、この問いは AI 支援開発において具体的になった[9,10]。しかし、別コードの生成と、そのコードを業務システムとして採用できることの間には検証が必要である。既知の例に合格する実装でも、妥当な国際文字列を拒否し、

不正な履歴を受け入れ、記載された件数より先に保存経路が破綻する可能性がある。代替実装を作る費用と、それを採用する正当性は別の研究課題である。

本稿は「業務主権」を、組織が業務上の約束を定義・承認し、それを体現するデータと履歴を保持し、検査可能な証拠に基づいて実装を選べる実務上の能力、という工学的目標として定義する。法的主権、あらゆる技術への依存の解消、実証済みの組織的成果を意味しない。今回の対象は、小さな構成的事例における技術上の成立条件である。

貢献は三点ある。第一に、動作検査、状態と履歴の保持、運用上限、権限境界を合わせた、条件付きの実装交換判定を示す。第二に、反例を契約と評価へ戻す仕様中心の開発・レビュー工程を具体化する。第三に、二実装の成果物を分析し、観察結果の支持範囲と限界を報告する。仕様先行、契約、移植可能性自体の発明を主張するものではない。

2 背景と関連研究

2.1 抽象化と意図の保持

Parnas の分割基準は、変化する設計判断をモジュール境界の内側へ置く[3]。Design by Contract は義務を明示し[4]、振舞い部分型は型の置換可能性に条件を与える[5]。本研究の交換判定はこれらを参照するが、有限の API 検査によって振舞い部分型や詳細化を証明するものではない。Leveson の意図仕様は、設計理由を含む仕様によって人間の問題解決を支える[6]。本研究も業務の理由を保持し、それに具体的な移行と証拠の工程を組み合わせる。

OMG の Model Driven Architecture は、業務機能を基盤技術から分離する[7]。Whittle らの産業調査では、モデル駆動による生成が選択的に使われる状況が報告されている[8]。本成果物は分離という目的を共有するが、確立されたモデル変換言語ではなく、自然言語の方法論、JSON 設定、HTTP 評価、個別の処理実装を用いる。JSON だけで業務の完全な操作的意味を記述しているわけではなく、汎用生成器も実証していない。

2.2 AI 支援開発と仕様駆動開発

HumanEval は記述からのプログラム合成における機能的正しさを評価し[9]、SWE-bench は既存リポジトリの issue に応じた修正を評価する[10]。それらの評価単位は、本稿の状態を持つシステムの交換とは異なる。本稿の数値を両ベンチマークと比較しない。

近年の仕様駆動開発には本提案と直接重なる議論がある。Piskala は仕様の厳密さを段階化する[11]。Diaz らは人間とエージェントの協働を統制する仕組みに仕様を位置づける[12]。Agarwal らは推奨される実践と実際の実践を調査し、意図と実装詳細の緊張を扱う[13]。本稿は版を特定したプレプリントを新しい関連研究として引用し、その主張を普遍的な有効性の確立した証拠とは扱わない。本研究の狭い貢献は、交換判定条件と、言語間の契約差、移行、運用境界を扱った具体的事例である。対象を絞った文献調査だけから、この組合せの先例がないとは断定しない。

2.3 オラクルと実証結果の解釈

テストには、観測した動作が要求に適合するかを判定するオラクルが必要である[14]。実装同士の比較は不一致の検出に役立つが、共通の誤解を排除しない。Knight と Leveson の複数版プログラムに関する研究は、別々に開発したという理由だけで故障の独立性を仮定できないことを示す[15]。本事例では二実装と評価が同じ AI 支援作業の文脈で作成されており、共通原因による誤りは特に重要な脅威である。

Runeson と Höst は、ソフトウェア工学の事例報告において文脈、証拠、限界を明示することを重視する[16]。本稿はその報告姿勢を参照する。ただし対象は構成的な技術事例と事後的な成果物分析であり、産業現場のフィールド研究や、開発方法論の対照実験ではない。

3 業務主権開発

3.1 継承資産と交換判定

本方法論は三種類の資産を保持する。業務の意図は、目的、責任、例外を関係者が読める言葉で記録する。契約は、観測可能な操作、表現、権限、保持すべき履歴、競合、容量を定める。証拠は、実際に検査した入力と版、その結果、未実施の評価を残す。実装ソースも重要な資産であるが、業務の意味を決める唯一の正本とはしない。

「交換判定」は、対象負荷と観測範囲を定め、五つの条件を記録して判断する手続きである。移行元と候補実装が選定した契約検査を満たすこと、移行が定めた業務状態を保持すること、監査履歴を各時点の仕様版とともに解釈できること、隠れた下位上限なしに記載された運用範囲を扱えること、export・import・仕様変更が明示した権限に従うことを要求する。成立の判断は、開示された検査と仮定に対して相対的であり、全入力や将来の全環境に対する証明ではない。

本成果物における保持対象は、申請 ID、属性、状態、起票情報、申請履歴、仕様版、仕様切替履歴である。workspace 固有の認証 ID、学習進捗、実装固有の revision、export 時刻は、業務移行の保持主張に含めない。評価 runner が直接比較するのは申請配列と仕様切替監査配列であり、export の全最上位項目を個別に比較してはいない。仕様版は import 処理で設定され、監査の整合検査を受ける。この差を踏まえて、記録された保持主張の強さを限定する。

3.2 開発とレビューの工程

最初に業務の目的と反例を採取し、状態、役割、通信表現、保持、競合、上限を契約にする。処理本体とは別ファイルに公開インターフェースの評価を準備する。AI を用いて実装と修正を行い、規則の採用と証拠の判定には人間が責任を持つ。別実装と移行によって同じ約束を検査し、不一致があれば仕様、評価、実装、教材を合わせて更新する。

この工程では契約の改訂を許す。初期評価を固定したまま、後から追加した全評価を独立した holdout として扱うわけではない。本事例ではレビューが仕様と最終 44 件評価の双方に影響した。したがって観察結果は、反復修正後の適合であり、未使用評価集合に対する盲検の一般化性能ではない。

3.3 AI の役割

AI は草案、実装、調査、是正を支援した。一方、保持された成果物には、凍結した生成手順、全実装工程について検証可能なモデル識別子、乱数 seed、比較可能な token 予算、完全な介入記録がない。このため、特定モデルの再生成能力ではなく、結果として得られた工学的成果物と方法を評価する。「別実装」とは言語ごとの処理が別々に書かれていることを指し、独立したチーム、異なるモデル提供元、相互のソース非閲覧を意味しない。

4 研究設計

4.1 研究課題と分析単位

RQ1 は、別々に実装した二つの API が、改訂された観測可能な業務契約を満たすかを問う。RQ2 は、選定した業務状態と履歴が双方向の移行と件数境界で保持されるかを問う。RQ3 は、この開発過程でどの不一致が従来の共通評価をすり抜け、交換判定の条件にどう関係したかを問う。

分析単位は、一つの合成的な設計変更申請アプリケーションと、その修正過程である。二言語は二つの独立した組織事例ではない。アサーション群も統計標本ではない。記述的な結果と分析上の含意を報告し、有意差検定、信頼区間、因果効果の推定、異種検査を合算した成功率は示さない。

4.2 成果物と環境

業務は、下書き起票、訂正、提出、承認、理由付き却下、取下げを持つ。管理テナントには設計者、承認者、設計兼承認者の三つの合成人物があり、別テナントにもう一人を置く。仕様 v1 は自己承認を許し、v2 は自己承認と自己却下を拒否する。拒否した更新は、本文、履歴、業務 revision を変更してはならない。

JavaScript 版のローカル環境は Node.js 26.5.1 と組込み SQLite である。公開版は Worker、D1、R2 を用いる。交換先は Python 3.14 の標準ライブラリ HTTP サーバーと SQLite である。両実装は同じ業務 JSON の自分用コピーを読む。ローカルは起動時、Worker はビルド時に取り込み、起動時に検査する。業務処理コードは共有しない。初期構想の Quarkus、FastAPI、PostgreSQL、Podman は、今回の実装環境には含まれない。

周辺成果物には、8 単元の学習サイト、NotebookLM で生成した 8 本の動画、DB スナップショットがある。これらは方法論の配布を助けるが、学習効果の証拠ではない。ローカル要求は開発用の固定 workspace を使う。ローカルの受け入れ評価は人物とテナントの規則を通るが、公開認証基盤を端から端まで検査してはいない。

4.3 証拠と来歴

評価記録の完了時刻は 2026-11-07T02:18:09Z、対象コードの基準 commit は `130054398f6c72c071fdeea266fcac714250fd1f` である。本稿が参照する後続の成果物スナップショットは `233e9e46592ba48b5686dae99c7c829edb31e09c` である。これらは開発履歴の commit であり、公開再現用の基準はタグ `artifact-v1.0.0` とする[17]。`content/evidence.json` の 8 入力

ハッシュが、仕様、二実装、runner、保存 adapter、受け入れ評価、画面コードを特定する。ハッシュ一致が示すのはバイト列の同一性であり、意味上の正しさではない。

提供された 13 項目のレビュー、是正記録、実際の runner、配信後の確認記録も調べた。以下で述べる修正前の反例はレビューに帰属させ、本稿のために旧実装を新しい対照条件として再実行してはいない。各 21 件合格という以前の記録は履歴上の観察であり、最終 44 件と対応づけた対照実験ではない。

4.4 手順と件数の解釈

runner は新しい loopback サーバーと DB を起動し、言語ごとに一度の受け入れ評価、各方向一つの小規模移行、各言語を移行元とする最大文字数の Unicode 下書き 200 件の生成、別実装への大容量移行を行う。また、JSON の文字数と部品を変更し、未対応状態の起動拒否、専用ディレクトリへの動画取込を検査する。終了時に自分が起動した子サーバーを停止する。

小規模移行の 6 件とは、二つの fixture のそれぞれについて、取込成功、申請配列一致、仕様切替監査配列一致を検査したアサーション数である。独立した 6 回の移行ではない。容量の 6 件も、各言語について、200 件で 1MiB を超えるデータの読出し、201 件目の拒否と状態保持、大容量移行と申請保持を検査した 3 群ずつである。処理量や応答時間の分布は記録していない。その他のモジュール検査、保存検査は範囲が異なるため分けて報告する。

5 契約と実装

5.1 表現と操作

タイトルは 120、理由は 2,000 の Unicode コードポイント以内とし、検査する文字列項目の不正なサロゲートを拒否する。端の空白除去に用いる文字も共通の一覧で定める。日時は実在するグレゴリオ暦の日付を持つ `YYYY-MM-DDTHH:mm:ss.SSSZ` の UTC 形式であり、年 0000 を拒否する。revision は 0 以上 9,007,199,254,740,991 以下の安全な整数値とする。JSON の真偽値、文字列、非整数値は拒否する。`1.0` のような JSON 数値表記は、解析後の値が許容する整数であれば受理する。契約は数値表記の字句ではなく、値についてのものである。

下書きの起票者かつ設計者だけが訂正・提出・取下げを行える。提出後や終端状態では下書き訂正を行えない。訂正では変更前後の属性を履歴に残す。取下げも申請と履歴を保持し、件数上限に含める。v2 の禁止規則はその後の承認・却下へ適用し、過去の履歴を書き換えない。

5.2 権限と監査

通常の一覧と個別操作はテナント内に限定する。別テナントで役割を持つだけでは workspace 全体の権限を得ない。管理テナントの承認者だけが、本人 workspace の全合成テナントを export・import し、仕様を切り替えられる。この例外によって、workspace 単位の移行を維持しつつ、その実行者を明示する。仕様切替は、操作者、時刻、変更前後の版を `policyHistory` に保存する。

import では、人物、テナント、起票・提出の役割、状態遷移、時系列、訂正属性、その時点の自己承認規則を検査する。これは契約に照らした内部整合性の確認であり、履歴の発生元の認証で

はない。契約上整合する架空のパックを作ることは可能である。署名による来歴保証や外部の証明は対象外である。

5.3 保存と運用範囲

上限は、保持申請 200 件、要求本文 24,000,000 バイト、workspace16,000,000 バイト、申請レコード 200,000 バイト、申請履歴 20 イベント、仕様切替履歴 2,000 イベントを同時に適用する。十進のバイト上限と、回帰検査で用いる二進の 1MiB を区別する。訂正時には提出と判断のために履歴 2 枠を確保する。最大件数と最大履歴の任意の組合せが格納できるという契約ではなく、各制約の論理積である。

修正前のローカル JavaScript 版は、Python を起動し、全状態をバッファ付き標準出力で受け取っていた。修正後は Node.js 内の SQLite を用いる。JavaScript の保存層はシリアライズした workspace を複数行に分け、条件付きトランザクションまたは D1 batch でメタデータ、分割片、revision を更新する。compare-and-swap で失敗した書込みが、成功側の分割片を変更してはならない。学習進捗は別テーブルと別 revision にし、確認問題の保存が業務操作を競合させないようにした。

これらは運用範囲を支える機構である。実行した 200 件負荷は、履歴の短い新規下書きであり、workspace と履歴の複合上限を尽くしていない。また、ローカル SQLite で SQL 保存層を検査しても、公開 D1 の全容量、競合、障害条件を確認したことにはならない。

6 結果

6.1 RQ1 観察した適合

各実装は 44 件の受け入れ評価すべてに合格し、失敗は 0 であった。正常起票、不正属性・役割、起票者だけの提出、v2 の自己承認拒否、拒否後の保持、別テナント拒否、別承認者の判断、v1 の比較動作、古い・不正な revision、却下理由、訂正と取下げ、import 制限、学習と業務の revision 分離を含む。

表現の反例も評価に含まれる。U+1F600 を 120 個含むタイトルは受理し、121 個は拒否する。非 ISO 日時、存在しない暦日、年 0000、不正サロゲート、設計者役割のない人物を起票者とする import を拒否する。ただし、これによって任意の Unicode 正規化の等価性、書記素クラスタ単位の上限、解析器全体の一致、全操作列の正しさを示したわけではない。

設定の 4 アサーションも合格した。各実装で文字数上限の変更と新部品の追加を合わせて確認し、未対応の状態構成で起動を拒否した。選定した JSON 設定が実行に反映される証拠であり、任意の自然言語規則からの自動実装や、全ての未対応仕様の拒否を示すものではない。

6.2 RQ2 検査した移行での保持

小規模移行は双方向とも成功した。各移行元で、補助平面の文字を最大文字数まで含むタイトルの申請を作り、訂正、取下げ、仕様切替を行った。6 アサーションが合格し、訂正と取下げを含

む申請配列および仕様切替監査配列が一致した。export 時の訂正後タイトルは初期タイトルより短いが、初期の最大文字数の値は起票・訂正履歴に残る。

各実装を移行元として、U+1F600 を使った 120 コードポイントのタイトルと 2,000 コードポイントの理由を持つ下書きを 200 件作った。export したデータは 1,048,576 バイトを超えた状態で読み出せ、別実装へ取り込んだ申請配列が一致した。201 件目は状態を変えずに拒否した。容量の 6 アサーション群はすべて合格した。記録しているのは閾値を超えたという検査であり、正確なパックサイズの実測値は報告しない。

実 SQLite 上の保存検査 5 件も合格した。同じ revision の同時更新は一回だけ成功し、1MiB 超の分割保存を往復でき、古い書込みが保存済み分割片を壊さず、再起動後も内容を保持し、別所有者は読めなかった。メモリ上の境界 4 件、メディア 7 件も合格した。ローカル動画取込 1 件では、専用ディレクトリへの保存と Git 管理動画の不変を確認した。17 資産のサイズ・ハッシュおよびスナップショット整合性と教材用 ID の複合検査 1 件も合格した。これらを合算した信頼性の比率は示さない。

6.3 RQ3 以前の評価が覆わなかった不一致

提供されたレビューは、以前の ASCII と通常の日付を中心とする評価から漏れた、三つの言語間差を報告した。JavaScript は UTF-16 コード単位、Python はコードポイントを数えていた。JavaScript は地域依存形式の import 日時を受理し、Python は拒否した。Python は真偽値 `true` と整数 revision 1 を等しいと扱い、JavaScript は更新を拒否した。これらは以前の成功例とは矛盾せず、交換の前提を破る。

レビューは、仕様内の 90 件による 1,169,346 バイトの状態が、ローカル子プロセスの出力上限を超え、宣言上は 200 件までであるのに `ENOBUFS` を生じたことも報告した。これは過去のレビュー観察であり、本稿で改めて計測した基準値ではない。修正後の負荷検査は 1MiB 超の処理成功を示すが、一般的な性能改善量を測定していない。

その他の指摘には、設定の転記、全体 export と仕様変更の過大な権限、仕様監査の欠如、下書き訂正・取下げの欠如、import 時の役割検査、学習と業務の revision 干渉、異なるビルド入口、誤解を招く評価表示、追跡動画の上書き、スナップショットの識別子、教材記録の不整合があった。一つの開発過程で得た診断であり、代表性を持つ欠陥分類や欠陥頻度の測定ではない。

6.4 公開環境での観察

配信後の確認は、Sites ソース commit `8ef0f63b71cbbad26feb7f2d797f825d3203e50d` を対象とした。検査入力のハッシュはローカル記録と一致した。認証済みブラウザーで、承認済み申請 1 件と履歴 3 件を読み、v2 から v1、v1 から v2 へ切り替え、操作者と時刻を持つ二つの仕様監査イベントが表示され、申請が保持されることを確認した。実証画面には 44 件の評価記録と対象 commit が表示された。

これは配信後の狭い読出し・書込み・表示経路の確認である。公開認証を通した受け入れ評価全体、公開 D1 の 200 件負荷、公開環境からの双方向移行を再実行したものではない。成果物と論文の双方でこの区別を保つ。

7 考察

7.1 インターフェース評価の合格だけでは交換を説明できない

本事例は、交換の根拠が項目名と状態コードだけでなく、文字列と時刻の意味を含む必要があることを示唆する。履歴と、状態を移送・保存する実際の経路も対象になる。仕様が 200 件を定めても、その手前で adapter がバイト数制約に達することがある。一覧をテナント別に制限していても、別の export 経路が全テナントを出力することがある。したがって、表現、権限、容量を交換判定の条件として明示する意義がある。

本方法論は、判断をどこに記録し、何を根拠に検査するかを変更する。実装や保守の作業そのものをなくすものではない。自然言語の規則には操作的意味が必要であり、JSON の読込みが解消するのは選定した設定の重複である。初期の仕様と評価が不完全だったからこそ、反例を戻す工程に意味がある。

7.2 仕様駆動開発への追加的知見

抽象化、契約、モデル駆動は、業務上の約束と技術的な実現を分離する根拠を既に与えている[2–8]。近年の SDD も意図の保持とエージェントの統制を扱う[11–13]。本稿が追加するのは、交換候補が、明示した権限と負荷の仮定の下で、特定の業務上の観測と履歴を保持するべきだという、具体的で反証可能な工学的主張である。export・import と容量の反例により、状態を持つ業務システムでその主張を検査可能にした。

成功の原因が AI であったこと、AI が不可欠であること、従来開発より優れることは結論できない。契約と通常の開発道具を用いたチームも、同じ評価を構成できる。AI は本事例の構築環境であり、代替実装を作る手段の候補である。その追加的便益は別の実証課題である。

7.3 組織的な決定権は今後の仮説である

成果物は、意図、契約、履歴、証拠を保持する技術的機構を提供する。しかし、それが実際の選択権につながるかは、技能、契約、配信環境へのアクセス、レビュー能力、移行費用に依存する。本研究では実組織や業務関係者の行動を測定していない。8 単元の教材もその代わりにはならない。業務主権は部分的に操作化した目標であり、この実証によって達成を確定した組織的成果ではない。

8 妥当性への脅威

構成概念妥当性。 選定した HTTP 評価への適合は、契約適合の近似であり、システム全体の正しさや組織の自由を測定しない。履歴の内部整合性は、認証された来歴より弱い。交換判定自体も、十分性を独立に検討すべき提案である。

内的妥当性。 仕様、二実装、評価は同じ AI 支援の文脈で改訂され、レビューが最終評価に影響した。共通の誤解と、既知欠陥への修正に向けた評価により、見かけの網羅性が高まる可能性がある。21 件と 44 件では測定手段が変わっており、対照条件を持つ前後比較ではない。

外的妥当性。 小さな合成ワークフロー、4 人物、2 言語、限定したデータモデルから、外部連携、実利用者の管理、長期のスキーマ進化、運用事故を持つ企業システムへ一般化できない。両実装がローカルでは SQLite を使い、保存技術の多様性も限られる。公開確認は D1 の一部経路に限る。

結論妥当性。 アサーションは互いに依存し、検査対象も異なる。欠陥確率や統計的な信頼性を推定しない。工数、遅延、費用、利用者成果の測定がないため、優越性は示せない。また、以前の反例を本稿のために独立に再現してはいない。

再現可能性。 コード、入力ハッシュ、コマンドは保持するが、AI 生成手順の凍結や独立した再実行はない。公開再現版は識別子を置換したスナップショットと固定タグを、新しい Git 履歴で提供する[17]。元の非公開履歴と未置換スナップショットは含まない。公開によって再実行は可能になるが、第三者による独立再現や AI 生成手順の再現を確定するものではない。

9 今後の研究

次の評価では、比較基準と交換判定を事前登録し、仕様と留保評価を実装前に固定し、相手実装を参照しない独立チームを用いるべきである。異なる AI 提供元と非 AI の比較条件を設け、介入と工数を記録し、初期構築と後続変更の双方を測定する。

移行では、生成した操作列、長い履歴、保存の複合境界、不正・架空パック、スキーマ変更、認証 ID の対応付け、障害注入、切替後の書込みと復旧を扱う。公開評価では認証と保存を端から端まで通す。フィールド研究では、業務関係者が規則を理解・承認でき、実際に実装を選び直せるかを測定する。実現可能性、AI の因果的寄与、組織的帰結を分けて検証することが必要である。

10 結論

本研究は、実装を選ぶ能力を支える仕様中心の方法論として業務主権開発を示した。一つの構成的ワークフローにおいて、二言語が各 44 件の受け入れ評価を満たし、検査した移行で申請履歴と仕様監査を保持した。レビューの反例から、その結果を表現の意味、権限、運用上限、証拠の来歴で限定する必要が明らかになった。結果は限定された工学的実証と研究計画を支持する。完全な実装独立、無人再生成、方法論の歴史的必然性を示すものではない。

データとソフトウェアの入手可能性

公開リポジトリ Business Sovereignty Replication Artifact は、ソース、仕様、保存済み評価、動画 8 本、識別子置換済み SQLite スナップショットを提供する[17]。コードとデータの基準は固定タグ artifact-v1.0.0、論文は `main/research/paper` を参照する。4.3 節の過去の開発 commit は公開の新しい履歴には含まれず、保持した入力ハッシュによって対応づける。ローカル再現には Node.js と Python を用い、クラウドアカウントや公開環境の認証情報を必要としない。配信済み実証サイトへのアクセス制限は継続する。元の非公開履歴と未置換スナップショットは公開版に含めない。

AI 支援と著者責任

AI は、成果物の構築と修正、文献探索、本稿の草案と翻訳を支援した。保持した記録から特定モデルの性能には帰属できない。人間の著者は、投稿前に本文、引用、配布権限、投稿先の AI 開示および著者要件を確認する責任を負う。所属、資金提供、利益相反の情報は提供されておらず、推定していない。本稿は投稿済み・採録済みの論文ではない。

付録 A 再現手順

公開の固定タグ `artifact-v1.0.0` を clone する[17]。開発時のスナップショット `233e9e46592ba48b5686dae99c7c829edb31e09c` は過去の来歴を示す識別子であり、公開版の checkout 先ではない。実行前に `content/evidence.json` の 8 入力ハッシュを照合する。Node.js 22.13 以上を必要とし、記録した実行は 26.5.1 である。Python 3 を用い、記録した実行は 3.14 である。リポジトリ直下で次を実行する。

```
git clone --branch artifact-v1.0.0 --depth 1 https://github.com/keijijin/business-sovereignty-reproducibility.git
cd business-sovereignty-reproducibility
node scripts/build.mjs
node scripts/validate-artifact.mjs
python3 scripts/review-checks.py
```

runner は独自の DB と loopback ポートを作り、新しい `content/evidence.json` を書き、子サーバーを停止する。再実行前に過去の記録を別途保持する。個別の範囲は `node scripts/check-storage.mjs`、`node scripts/check-boundaries.mjs`、`node scripts/check-media.mjs`、`python3 scripts/verify-snapshots.py` で確認できる。これらから性能 SLO への適合を推定しない。サイトのソース保存基盤は最大の動画オブジェクトを保持できないため、公開ソースは別履歴を持つ。評価成果物との対応は、リポジトリ全体の commit 一致ではなく、業務入力のハッシュによって示す。

参考文献

[1] G. M. Amdahl, G. A. Blaauw, and F. P. Brooks, Jr. (1964). Architecture of the IBM System/360. IBM J ournal of Research and Development, 8(2), 87–101. Source.

[2] E. F. Codd (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the A CM, 13(6), 377–387. Source.

[3] D. L. Parnas (1972). On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM, 15(12), 1053–1058. Source.

[4] B. Meyer (1992). Applying Design by Contract. Computer, 25(10), 40–51. Source.

[5] B. Liskov and J. M. Wing (1994). A Behavioral Notion of Subtyping. ACM Transactions on Programming Languages and Systems, 16(6), 1811–1841. Source.

[6] N. G. Leveson (2000). Intent Specifications: An Approach to Building Human-Centered Specifications. IEEE Transactions on Software Engineering, 26(1), 15–35. Source.

[7] Object Management Group (2014). Model Driven Architecture Guide Revision 2.0. OMG document ormsc/2014-06-01. Source.

[8] J. Whittle, J. Hutchinson, and M. Rouncefield (2014). The State of Practice in Model-Driven Engineering. IEEE Software, 31(3), 79–85. Source.

[9] M. Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374v2. Source.

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. International Conference on Learning Representations; arXiv:2310.06770v3. Source.

[11] D. B. Piskala (2026). Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. Preprint, arXiv:2602.00180v1. Source.

[12] J. Diaz, J. Gayoso, A. Cimminio, and J. Perez (2026). Spec-Driven Development for Agentic Software Engineering: Harnessing Human–Agent Teamwork. Preprint, arXiv:2609.00252v1. Source.

[13] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux (2026). Spec-Driven Development a s Prescribed and as Practiced. Preprint, arXiv:2608.25202v4. Source.

[14] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo (2015). The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering, 41(5), 507–525. Source.

[15] J. C. Knight and N. G. Leveson (1986). An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. IEEE Transactions on Software Engineering, SE-12(1), 96–109. Source.

[16] P. Runeson and M. Höst (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. Empirical Software Engineering, 14(2), 131–164. Source.

[17] K. Jin (2026). Business Sovereignty Replication Artifact, version 1.0.0. Source, specifications, evaluations, videos, and sanitized database snapshots. Fixed baseline.